Quickdress: Add Item Interface Design

Capturing an item with the camera or choosing it from the photo library · Sprint 1 · November 6, 2026

Prepared by Marwan Abouelfotuh

Team: Aya Noaman, Malk Metwally, Nada Salem, Marwan Abouelfotuh, Omar Hassan

This document designs the screens a user goes through to add a piece of clothing: choosing the camera or the photo library, giving permission, cutting the garment out of its photo, entering its details and saving it. It covers FR-01 (add an item by photo or from the library) and FR-02 (store its attributes), and it is shaped by NFR-04 (first outfit within 5 minutes), NFR-07 (permissions on first use, app usable without them) and NFR-02 (data stays on the phone).

It also fixes the visual look of the whole app, because the add-item screens are the first screens built in that look. Every screen shown here is a concept mock built from real layout code at iPhone size. Nothing in this document has been tested with users yet.

Decisions at a glance

Decision	Choice	Why
Look of the app	The claim ticket: each garment carries a numbered paper tag, like clothes at a dry cleaner or ironing shop	Chosen from three directions (section 1). It gives every item an identity and turns adding clothes into one familiar ritual.
Details screen	A receipt under the garment, one line per attribute, saved by tearing off a stub	Chosen from three layouts. It shows every attribute at once and makes the missing one obvious.
Camera screen	Apple's own camera (`UIImagePickerController`)	Matches the Design Specification, needs no camera code of our own in Sprint 1, and users already know it.
Library screen	Apple's `PhotosPicker`	Needs no Photos permission at all, so the library always works, even when the camera is denied.
Background removal	Cut the garment out on the phone with Apple's Vision framework	Real photos are taken on beds and floors. A cut-out makes every screen, and the future colour suggestion (FR-13), work with messy photos. Nothing leaves the phone.
Defaults	Warmth, formality and comfort start at 3. Category, colour and season have no default.	Three answers are needed instead of six, which keeps a new user inside NFR-04. A wrong category or colour would break the outfit rules, so those are always asked.
Item numbers	Each item gets the next number (No. 032) when it is saved, and numbers are never reused	The tag number is how the item is named on every screen. It needs a small change to the core (section 6).

Table 1. Main design decisions.

1. The look

1.1 Where it comes from

Three looks for the home screen (a TV weather forecast, claim tickets and airport signage) were generated as concept images with the same content, and the claim ticket was chosen. Three layouts for the add-item details screen in that look (a receipt, a large tag, and a clothes rail) were then compared, and the receipt was chosen. The rest of the team still needs to agree to the look (section 8). Figure 1 shows the two approved concept images. They set the look; the mocks in section 3 are the exact layouts.

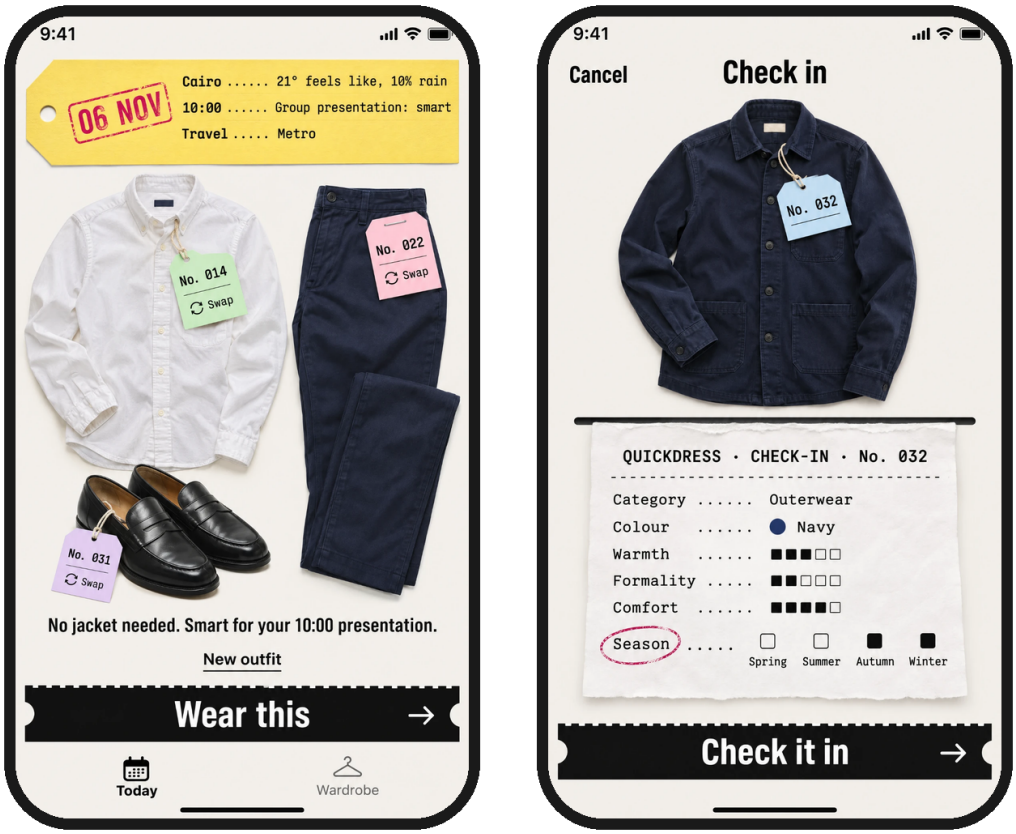


Figure 1. Approved concept images: the Today screen (left) and the check-in screen (right). Generated images, used as the visual reference only.

1.2 Colours

The ground is a warm off-white, like a shop counter. Colour is used for meaning, not decoration: each category has its own tag colour on every screen, and the red-violet stamp ink only marks the one thing that needs the user's attention.

Name	Value	Used for
Ground	#F3F1EC	The background of every screen.
Ink	#141414	Text, rules, filled level squares, the main button (the stub).
Secondary ink	#57534C	Supporting text. 6.8:1 on the ground.
Receipt paper	#FCFBF8	The receipt and the add-sheet tickets.
Stamp ink	#C2185B	The ring around the line that needs an answer, the camera-off mark, and the Checked in stamp. Never used for anything else.

Name	Value	Used for
Tag: top	#BDE8D2 (mint)	Tags on tops, everywhere in the app.
Tag: bottom	#F7B9C8 (pink)	Tags on bottoms.
Tag: outerwear	#BFD9F2 (sky)	Tags on outerwear. New in this document.
Tag: shoes	#D9C6F0 (lilac)	Tags on shoes.
Tag: accessory	#F9C9A6 (peach)	Tags on accessories. New in this document.
Tag: not yet sorted	#FCFBF8 (blank)	The tag before a category is chosen.
Day ticket	#F6E27A (canary)	The day's ticket on the Today screen only.
Button off	#D9D5CD with #5F5B54 text	The stub while something is still missing. 4.6:1.

Table 2. Colours. In code these are named colours in the asset catalogue, with a dark-mode variant for each to be designed in Sprint 2.

1.3 Type

Role	Face	Where
Statements	Pragati Narrow Bold	Screen titles, the stub label, sheet headings, text buttons. A condensed bold face measured from the approved concept image.
Ticket text	Chivo Mono	Everything printed on a receipt, tag or ticket: attribute names, values, item numbers, small notes.
Reading text	SF Pro (system)	Sentences of explanation, and Apple's own controls and alerts.

Table 3. Type. Pragati Narrow and Chivo Mono are free under the SIL Open Font Licence and are bundled with the app. All three follow Dynamic Type through `UIFontMetrics`.

1.4 Building blocks

Block	What it is	Where it appears
Tag	A paper tag with a punched hole, tied to the garment by a string, showing the item number. Its colour is the category colour.	On every garment in the app.
Receipt	White paper with torn edges coming out of a thin printer slot. Lines are set in Chivo Mono with dotted leaders between name and value.	Check-in and checked-in screens.
Stamp ring	A hand-drawn oval in stamp ink around one receipt line.	The line that needs an answer now.
Stub	A black strip with a perforated top edge and notched ends. It is the one main action on a screen.	Check it in, Check in another, Continue, Wear this (Today).
Ticket	A paper ticket with a perforation line, used as a large tappable choice.	The add sheet.
Stamp	A rubber stamp impression, slightly rotated.	Checked in. Today will reuse it for Worn.

Table 4. Building blocks shared by the add-item screens and the rest of the app.

2. The flow

Figure 2 shows every path from the Wardrobe to a saved item. Boxes with a dashed border are Apple's own screens. They keep Apple's look because iOS does not allow them to be restyled, and users already know them. The two blue boxes are steps in the core that have no screen of their own.

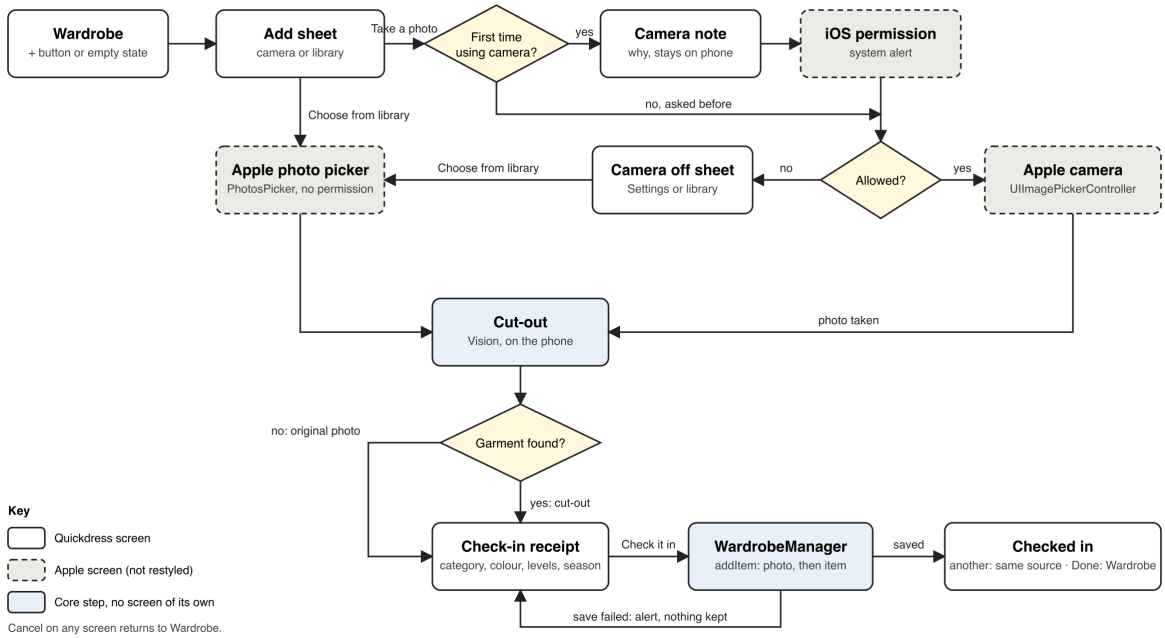


Figure 2. Flow for adding an item (FR-01, FR-02, NFR-07).

Three rules hold everywhere in the flow:

- **Cancel always works.** It returns to the Wardrobe. If a photo has already been taken, the app first asks "Discard this piece?" so a slip of the thumb does not lose a photographed item.
- **Nothing is saved until Check it in.** The photo and the item are written together by `WardrobeManager.addItem`, which already deletes the photo again if saving the item fails.
- **The library is always available.** Choosing a photo needs no permission, so a user who refuses the camera can still build a whole wardrobe (NFR-07). On the Simulator the camera option is not shown at all, because `CameraSource` reports itself unavailable there (risk R10).

3. Screens

3.1 Starting: the Wardrobe and the add sheet

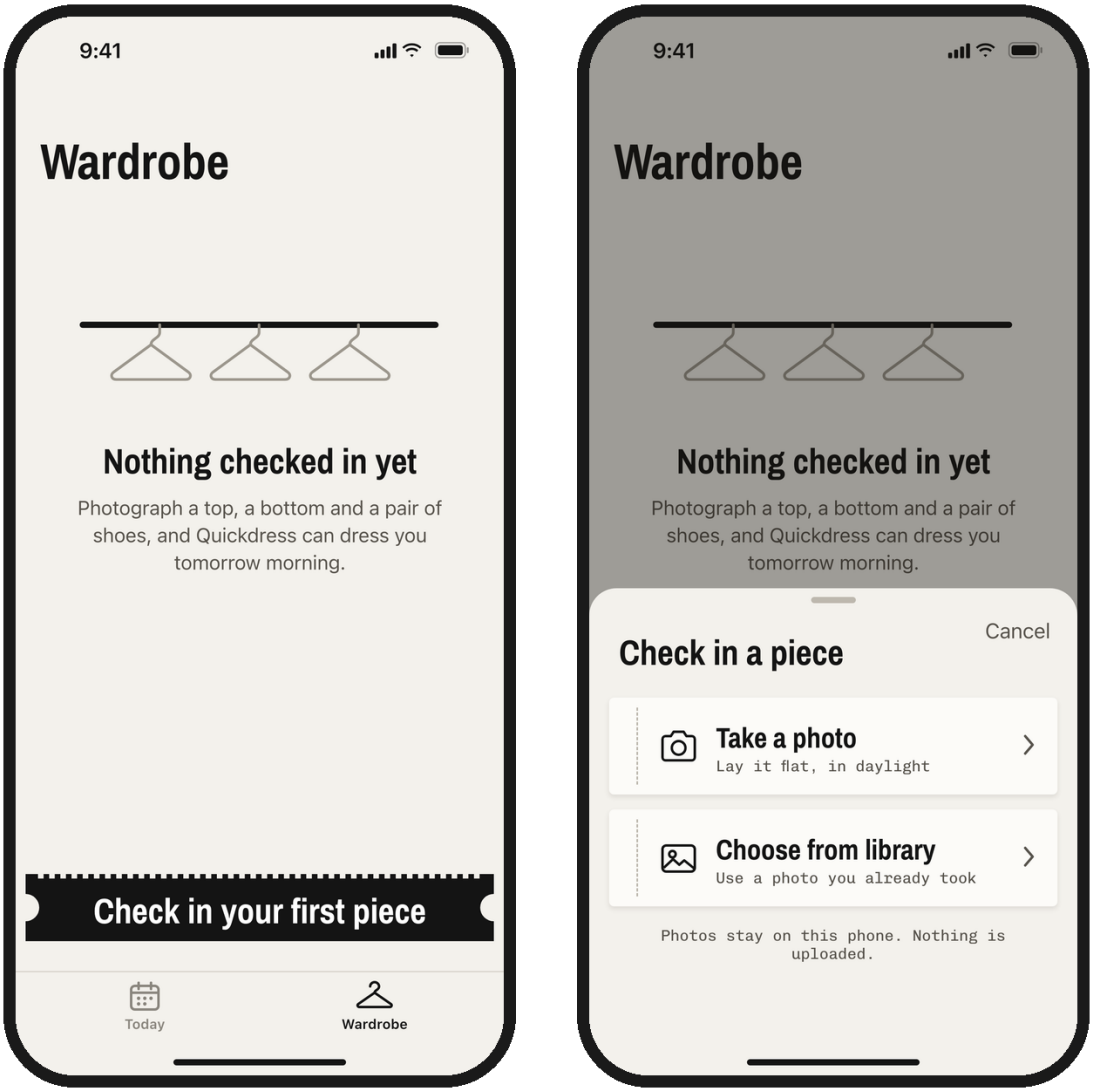


Figure 3. Empty Wardrobe (left) and the add sheet (right).

An empty wardrobe is the first thing a new user sees in the Wardrobe tab, so it explains what to do instead of showing a blank grid. When the wardrobe has items, the stub is replaced by a + button in the navigation bar, as on Omar's current Wardrobe screen. Both open the add sheet.

Element	Behaviour and exact text
Empty state	Title "Nothing checked in yet". Text: "Photograph a top, a bottom and a pair of shoes, and Quickdress can dress you tomorrow morning." These are the three pieces an outfit needs (FR-05), so the text doubles as the first-run goal for NFR-04.
Stub	"Check in your first piece". Opens the add sheet.
Add sheet	A sheet at medium height with the title "Check in a piece" and a Cancel button. One ticket per available image source, listed by the plug-in registry: "Take a photo" ("Lay it flat, in daylight") and "Choose from library" ("Use a photo you already took").

Element	Behaviour and exact text
Footer	"Photos stay on this phone. Nothing is uploaded." This is true for every source and answers the question users ask before they allow the camera.

Table 5. Entry screens.

3.2 Camera permission

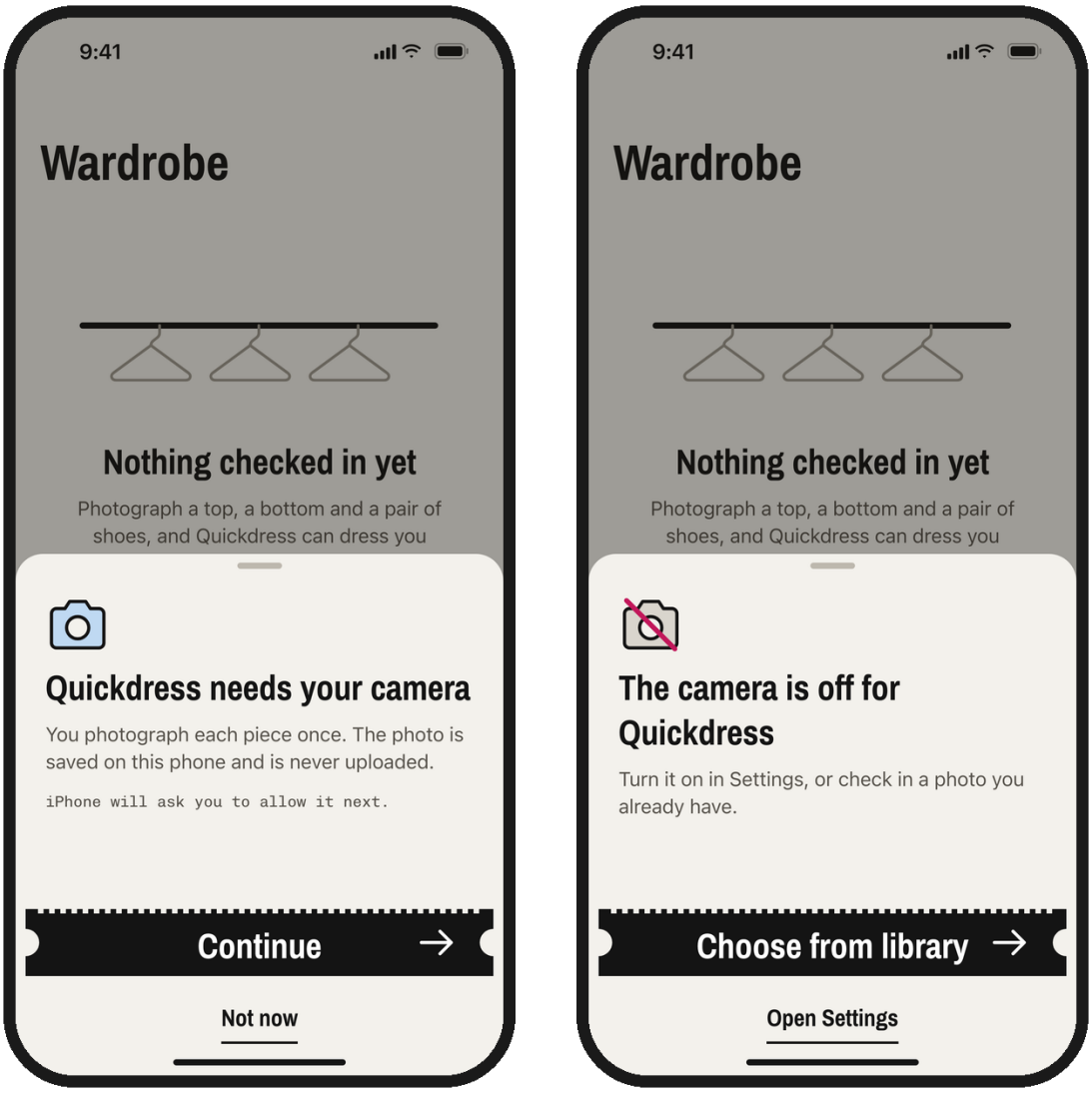


Figure 4. Camera note before the first request (left) and the sheet shown when the camera is off (right).

iOS shows its permission alert only once. If the user refuses it without understanding why the app needs the camera, getting it back means a trip to Settings. So the first time the user taps Take a photo, a short note explains the reason before iOS asks. Tapping Continue calls `requestAccess()` on `CameraSource`, which shows Apple's alert.

Case	What happens
First time	Camera note: "Quickdress needs your camera". "You photograph each piece once. The photo is saved on this phone and is never uploaded." Then "iPhone will ask you to allow it next." Buttons: Continue (stub), Not now (returns to the add sheet).
Allowed	Apple's camera opens straight away, now and every later time.

Case	What happens
Refused, now or earlier	Camera off sheet: "The camera is off for Quickdress". "Turn it on in Settings, or check in a photo you already have." The main action is Choose from library, because it works immediately. Open Settings opens the app's page in Settings.
Text in the iOS alert	Set through `NSCameraUsageDescription`: "Quickdress uses the camera to photograph your clothes. Photos stay on this iPhone."

Table 6. Camera permission cases (NFR-07).

3.3 Apple's camera and photo picker

The camera is Apple's `UIImagePickerController` with the rear camera, photo mode only and no editing step, because our own cut-out replaces cropping. The library is Apple's `PhotosPicker`, limited to images and to one photo at a time. Both return an image, which `CameraSource` and `PhotoLibrarySource` turn into JPEG data scaled so the long edge is at most 1,600 pixels, as the Design Specification already states. Neither screen is restyled.

3.4 Cutting it out

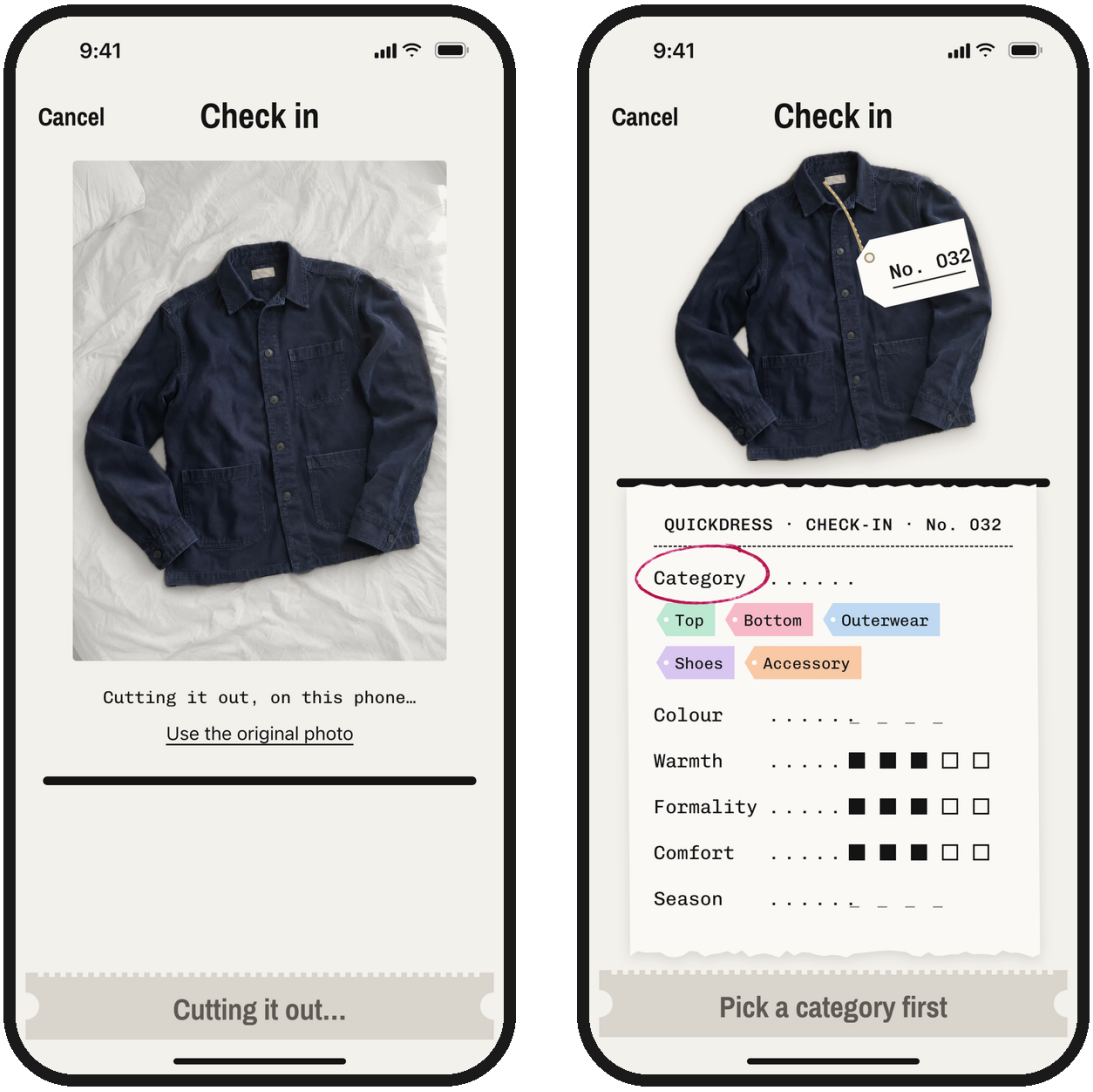


Figure 5. The cut-out running over the dimmed photo (left), and the receipt that follows, asking for the category first (right).

Right after the photo arrives, the app separates the garment from its background with Apple's Vision framework (`VNGenerateForegroundInstanceMaskRequest`, available from iOS 17, which is already the app's minimum). It runs on the phone, so nothing is uploaded and it works offline. While it runs, the photo is shown dimmed with the line "Cutting it out, on this phone...". A link, "Use the original photo", skips the cut-out.

We tried this on a realistic phone photo of a navy overshirt on an unmade bed (Figure 6). Vision found one subject and returned a clean cut-out in 3.4 to 3.8 seconds on a Mac, including loading the model the first time. The time on an iPhone is not measured yet; it is the first test in section 7. If Vision finds no subject, or the user prefers the original, the original photo is kept and everything else works the same.



Figure 6. A real test of the cut-out: the phone photo (left) and what Vision returned (right).

The saved photo is the cut-out, stored as a PNG because JPEG cannot hold transparency. This needs one small change to `FilePhotoStore`, which currently names every photo `.jpg` (section 6).

3.5 The check-in receipt

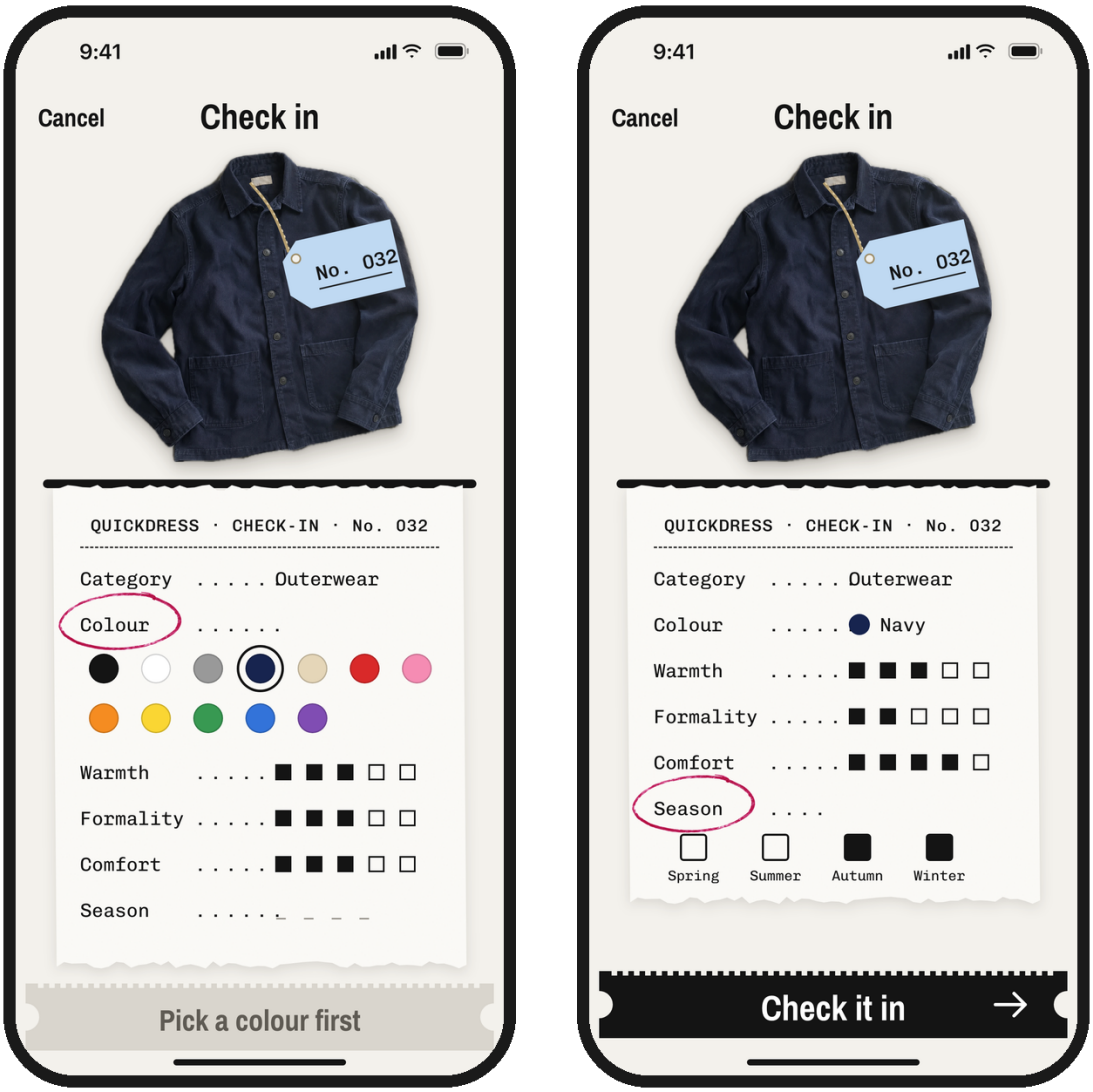


Figure 7. The receipt asking for the colour (left) and complete, with the season being answered (right).

The receipt is the heart of the flow. The garment sits at the top with its new tag, and the receipt underneath lists the six attributes from FR-02. The rules:

These rules match the validation already in the core: ratings from 1 to 5 and at least one season. Category and colour are not optional in `ClothingItem`, so the receipt makes sure they are chosen rather than filled with a silent default.

3.6 Checked in



Figure 8. After saving: the receipt is stamped and the next piece is one tap away.

When Check it in succeeds, the stub tears off, the receipt is stamped CHECKED IN and a line prints the date and time. The title changes to "Checked in" with Done in the top right, which returns to the Wardrobe. The stub now says "Check in another" and reopens the source the user just used, so photographing a top, a bottom and shoes in a row takes three short loops. That loop is what keeps a new user inside the 5 minutes of NFR-04.

3.7 Errors and edge cases

Situation	What the user sees
Saving fails (storage error)	An alert, "Couldn't check this in", "Nothing was saved. Try again, or cancel and take the photo again." Buttons: Try Again, Cancel. The receipt keeps every answer. `WardrobeManager` has already removed the photo, so no orphan file is left.
The photo can't be read	An alert, "This photo can't be used", "Try another photo." The user returns to the add sheet.
No garment found	The original photo is used without any message, and the cut-out link is hidden.
Cancel after the photo	A confirmation, "Discard this piece?", with Discard (destructive) and Keep editing.
The app is closed during check-in	Nothing is saved, as with Cancel. Check-in takes well under a minute, so restoring a half-finished receipt is not worth the extra state.
Simulator	Only Choose from library is listed.

Table 7. Errors and edge cases.

4. Motion and feedback

Motion is kept to the moments that carry meaning. Each one is short and has a plain replacement when Reduce Motion is on.

Moment	Motion	With Reduce Motion
Answer a line	The answer types into the line from left to right in about 0.15 s, then the ring slides to the next empty line.	The answer appears and the ring moves without sliding.
Check it in	The stub slides off to the right as if torn, the stamp drops onto the receipt (from 115% to 100% scale in 0.18 s), and the phone gives a success haptic.	The stamp fades in. The haptic stays.
Cut-out	The cut-out fades in over the dimmed photo, then the photo fades away.	Cross-fade only.
Errors	A warning haptic with the alert.	Same.

Table 8. Motion.

5. Accessibility

Area	How the design handles it
Contrast	Ink on the ground is 16.3:1, ink on the receipt 17.8:1, secondary text 6.8:1, the grey stub's text 4.6:1, white on the black stub 18.4:1 and the stamp ink on paper 5.7:1. All pass WCAG AA. The blank marks on unanswered lines are 3.4:1; they are decoration, and the line's name carries the meaning.
Touch targets	Receipt lines are drawn 37 points tall but their tap area is 44 points. Swatches, tags and tick boxes are at least 44 by 44 points including spacing.
VoiceOver	Each receipt line is one element read as name, value and state, for example "Warmth, 3 of 5, adjustable". Swiping up or down changes a rating without opening it. The ring is announced as "needs an answer". Swatches read their colour name. The garment image reads "Navy outerwear, number 32".
Dynamic Type	All text scales. At the largest sizes the receipt lines wrap the value under the name and the dotted leaders are hidden.
Colour	No information is carried by colour alone: category tags also carry the category name in the receipt, and every swatch has its name.

Table 9. Accessibility (NFR-09 is good practice this term, not formally tested).

6. How it maps to the code

The design fits the microkernel architecture without changing any contract except the one it introduces. The changes below are against the team repository as of November 6, 2026.

Where	Change
`QuickdressCore/Contracts/ImageSource.swift` (new)	The capture contract from the Design Specification: `isAvailable`, `requestAccess() async -> Bool`, `capture() async throws -> Data`, plus `id` and `priority` from `Plugin`.
`QuickdressCore/Domain/ClothingItem.swift`	Add `number: Int`, the tag number. `WardrobeManager.addItem` gives each new item the highest existing number plus one. Numbers are never reused, so a deleted No. 031 does not come back. **Needs Omar's agreement**, since he owns the core types.
`CapturePlugins` package (new)	`CameraSource` (wraps `UIImagePickerController`, unavailable on the Simulator), `PhotoLibrarySource` (wraps `PhotosPicker`), and `SubjectLifter`, a helper that runs the Vision cut-out. `SubjectLifter` is a helper like `ItemAnalyzer`, not a plug-in contract, so Table 3 of the Design Specification gains one row.
`StoragePlugins/FilePhotoStore.swift`	Name files by their real type (`.png` for cut-outs, `.jpg` otherwise) instead of always `.jpg`.
`Quickdress/AddItem/` (new)	`AddItemSheet`, `CameraNoteSheet`, `CameraOffSheet`, `CheckInView` and `CheckInViewModel` (MVVM, as the other screens), plus the building blocks `TagView`, `ReceiptView`, `StubButton` and `StampView`.
`Quickdress/Theme/` (new)	Named colours (Table 2), the two bundled fonts with their licences, and the two stamp images.
`project.yml`	`INFOPLIST_KEY_NSCameraUsageDescription` with the text in Table 6, the `CapturePlugins` package, and `UIAppFonts` for the two fonts.
`Quickdress/Wardrobe/` (Omar's)	A + button in the toolbar and the empty state in Figure 3. Restyling the grid and item detail in the ticket look is a separate task.

Table 10. Code changes needed to build this design.

7. How we will test it

What	How	Pass mark
Cut-out speed	Time `SubjectLifter` with os_signpost on the oldest iPhone the team has, over 10 photos.	Under 2 s each, after the first.
Cut-out quality	Photograph 10 real garments from team members' wardrobes on beds, floors and chairs.	At least 8 of 10 usable without "Use the original photo".
Camera refused	Refuse the camera on a real iPhone, then add an item.	The camera-off sheet appears and the item is added from the library (NFR-07).
Simulator	XCTest: `CameraSource.isAvailable` on the Simulator.	False, and the add sheet lists only the library.
Validation	XCTest on `CheckInViewModel`: the stub label for each missing answer, and that Check it in is disabled until all are given.	All cases pass.
Save failure	Existing `WardrobeManagerTests` with a failing item store.	No photo file is left behind.
First pieces	Five people from outside the team add a top, a bottom and shoes on a phone with an empty wardrobe and no instructions.	Median under 3 minutes for all three, leaving room inside NFR-04's 5 minutes.
VoiceOver	Add an item with VoiceOver on, screen hidden.	Possible without sighted help.

Table 11. Tests for the add-item flow. The first two run in Sprint 1, the user test with the NFR-04 test at the end of Sprint 2.

8. Open questions for the team

- **The look itself.** The claim-ticket look was chosen while designing this screen. The team should agree to it before other screens are built in it.
- **Item numbers in the core.** Omar, are you happy to add `number` to `ClothingItem` and have `WardrobeManager` assign it?
- **Restyling the existing screens.** The Wardrobe grid and item detail are in the standard iOS style. Who restyles them in the ticket look, and in which sprint?
- **Two new tag colours.** Outerwear sky blue and accessory peach were added in this document. They need the team's agreement before they spread to other screens.
- **Bundled fonts.** Two font files (about 0.5 MB together) are added to the app. Everyone should be fine with that before the first pull request.
- **Design Specification.** Table 3 of the revised Design Specification needs a row for `SubjectLifter`, and the domain model needs the item number. These are small edits to make together with this work.